## SCJP 1.6 (CX-310-065 , CX-310-066)

### Subject: Thread and Concurrency

**Total Questions : 61**

**Prepared by : http://www.javacertifications.net**

SCJP 6.0: Thread and Concurrency
Questions Question - 1

**What will happen when you attempt to compile and run the following code ?**

```
1. public class Test extends Thread{
2.     public static void main(String argv[]){
3.         Test  b = new Test();
4.         b.start();
5.     }
6.     public void run(){
7.         System.out.println("run");
8.     }
9.     public void run(String s){
10.         System.out.println("run(String s)");
11.     }
12. }
```

1.run
2.run(String s)
3.run run(String s)
4.Compilation fails due to an error on line 9

Explanation :
A is the correct answer.

overloaded run(String s) method will be ignored by the Thread class unless you call it manually. The Thread class expects a run() method with no arguments. So run() will be called.

Question - 2

**What will happen when you attempt to compile and run the following code ?**

```
class A  implements Runnable{
    public void run(){
        System.out.println("run-A");
    }
}

1. public class Test {
```

```
2.     public static void main(String argv[]){
3.       A a = new A();
4.       Thread t = new Thread(a);
5.       t.start();
6.     }
7.     public void run(){
8.         System.out.println("run-Test");
9.     }
10. }
```

1.run-A
2.run-Test
3.run-A run-Test
4.Compilation fails due to an error on line 7

Explanation :
A is the correct answer.

method run() in Test class is a method not realted to thread. t.start() method calls run() method of class A.

Question - 3

```
What will happen when you attempt to compile and run the following
code ?

class A  implements Runnable{
    public void run(){
        System.out.println("run-A");
    }
}

1. public class Test {
2.     public static void main(String argv[]){
3.       A a = new A();
4.       Thread t = new Thread(a);
5.      System.out.println(t.isAlive());
6.      t.start();
7.      System.out.println(t.isAlive());
8.     }
9. }
```

1.false run-A true
2.false run-A false
3.true run-A true
4.Compilation fails due to an error on line 7

Explanation :
A is the correct answer.

Once the start() method is called, the thread is considered to be alive.

Question - 4

**What will happen when you attempt to compile and run the following code ?**

```
1. public class Test extends Thread{
2.    public static void main(String argv[]){
3.      Test t = new Test();
4.      t.run();
5.      t.start();
6.    }
7.    public void run(){
8.      System.out.println("run-test");
9.    }
10. }
```

1.run-test run-test
2.run-test
3.Compilation fails due to an error on line 4
4.Compilation fails due to an error on line 7

Explanation :
A is the correct answer.

t.run() Legal, but does not start a new thread , it is like a method call of class Test
BUT t.start() create a thread and call run() method.

Question - 5

**What is the output for the below code ?**

```
class A implements Runnable{
    public void run(){
        System.out.println("run-a");
    }
}

1. public class Test {
2.  public static void main(String... args) {
3.      A a = new A();
4.      Thread t = new Thread(a);
5.      t.start();
6.      t.start();
7.  }
8. }
```

1.run-a
2.run-a run-a
3.Compilation fails with an error at line 6
4.Compilation succeed but Runtime Exception

Explanation :
D is the correct answer.

Once a thread has been started, it can never be started again. 2nd time t.start() throws java.lang.IllegalThreadStateException.

Question - 6

```
What is the output for the below code ?

class A implements Runnable{
    public void run(){
        System.out.println(Thread.currentThread().getName());
    }
}

1. public class Test {
2.  public static void main(String... args) {
3.      A a = new A();
4.      Thread t = new Thread(a);
5.      t.setName("good");
6.      t.start();
7.  }
8. }
```

1.good
2.null
3.Compilation fails with an error at line 5
4.Compilation succeed but Runtime Exception

Explanation :
A is the correct answer.

Thread.currentThread().getName() return name of the current thread.

Question - 7

```
1. class A implements Runnable{
2.  public void run(){
3.      try{
4.          Thread.sleep(1000*5);
5.      // insert code
6.      }
7.  }
8. }

public class Test {
    public static void main(String... args) {
        A a = new A();
        Thread t = new Thread(a);
        t.setName("good");
        t.start();
```

```
    }
}
```

**Which of the following code to be inserted at line no 5 to compile withour error ?**

1.}catch(InterruptedException e){
2.}catch(IOException e){
3.}catch(ThreadException e){
4.}catch(IllegalThreadStateException e){

Explanation :
A is the correct answer.

sleep() method of Thread class throws InterruptedException.

Question - 8

**From the below code , thread will sleep for _____ ?**

```
class A implements Runnable{
    public void run(){
        try{
            Thread.sleep(1000*5);
        }catch(InterruptedException e){

        }
    }
}
public class Test {
    public static void main(String... args) {
        A a = new A();
        Thread t = new Thread(a);
        t.setName("good");
        t.start();

    }
}
```

1.5 seconds
2.5 mili seconds
3.5 minutes
4.5 hours

Explanation :
A is the correct answer.

Thread.sleep(long millis); sleep() method takes miliseconds , So 1000*5 = 5 seconds.

Question - 9

What is the output for the below code ?

```
class A implements Runnable{
    public void run(){
        System.out.println(Thread.currentThread().getName());
    }
}


public class Test {
    public static void main(String... args) {
        A a = new A();
        Thread t = new Thread(a);
        Thread t1 = new Thread(a);
        t.setName("t");
        t1.setName("t1");
        t.setPriority(10);
        t1.setPriority(-3);
        t.start();
        t1.start();

    }
}
```

1.t t1
2.t1 t
3.t t
4.Compilation succeed but Runtime Exception

Explanation :
D is the correct answer.

Thread priorities are set using a positive integer, usually between 1 and 10.
t1.setPriority(-3); throws java.lang.IllegalArgumentException.

Question - 10

What is the output for the below code ?

```
class A  implements Runnable{
    public void run(){
        try{
        for(int i=0;i<4;i++){
            Thread.sleep(100);
            System.out.println(Thread.currentThread().getName());
        }
        }catch(InterruptedException e){

        }
    }
}


public class Test {
    public static void main(String argv[]) throws Exception{
      A a = new A();
```

```
        Thread t = new Thread(a,"A");
        Thread t1 = new Thread(a,"B");
        t.start();
        t.join();
        t1.start();
         }
}
```

1.A A A A B B B B
2.A B A B A B A B
3.Output order is not guaranteed
4.Compilation succeed but Runtime Exception

Explanation :
A is the correct answer.

t.join(); means Threat t must finish before Thread t1 start.

Question - 11

```
What is the output for the below code ?

public class B {
    public synchronized void printName(){
        try{
                System.out.println("printName");
                 Thread.sleep(5*1000);

            }catch(InterruptedException e){
            }
    }

    public  synchronized void printValue(){
               System.out.println("printValue");
    }
}



public class Test extends Thread{
    B b = new B();
    public static void main(String argv[]) throws Exception{
    Test t = new Test();
    Thread t1 = new Thread(t,"t1");
    Thread t2 = new Thread(t,"t2");
    t1.start();
    t2.start();
    }

    public void run(){
        if(Thread.currentThread().getName().equals("t1")){
            b.printName();
        }else{
            b.printValue();
        }
```

```
    }

}
```

1.print : printName , then wait for 5 seconds then print : printValue
2.print : printName then print : printValue
3.print : printName then wait for 5 minutes then print : printValue
4.Compilation succeed but Runtime Exception

Explanation :
A is the correct answer.

There is only one lock per object, if one thread has picked up the lock, no other thread can pick up the lock until the first thread releases the lock. printName() method acquire the lock for 5 seconds, So other threads can not access the object. If one synchronized method of an instance is executing then other synchronized method of the same instance should wait.

Question - 12

```
What is the output for the below code ?

public class B {
    public synchronized void printName(){
        try{
               System.out.println("printName");
                Thread.sleep(5*1000);

            }catch(InterruptedException e){
            }
    }

    public   void printValue(){
              System.out.println("printValue");
    }
}



public class Test extends Thread{
    B b = new B();
    public static void main(String argv[]) throws Exception{
    Test t = new Test();
    Thread t1 = new Thread(t,"t1");
    Thread t2 = new Thread(t,"t2");
    t1.start();
    t2.start();
    }

    public void run(){
        if(Thread.currentThread().getName().equals("t1")){
            b.printName();
        }else{
            b.printValue();
```

```
        }
    }

}
```

1.print : printName , then wait for 5 seconds then print : printValue
2.print : printName then print : printValue
3.print : printName then wait for 5 minutes then print : printValue
4.Compilation succeed but Runtime Exception

Explanation :
B is the correct answer.

If a class has both synchronized and non-synchronized methods, If one synchronized method of an instance is executing, other threads can still access the class's non-synchronized methods. In this case printValue() method is not synchronized, So both method can run simultaneously.

Question - 13

```
What is the output for the below code ?

public class B {
    public static synchronized void printName(){
        try{
                System.out.println("printName");
                 Thread.sleep(5*1000);

            }catch(InterruptedException e){
            }
    }

    public synchronized void printValue(){
                System.out.println("printValue");
    }
}



public class Test extends Thread{
    B b = new B();
    public static void main(String argv[]) throws Exception{
    Test t = new Test();
    Thread t1 = new Thread(t,"t1");
    Thread t2 = new Thread(t,"t2");
    t1.start();
    t2.start();
    }

    public void run(){
        if(Thread.currentThread().getName().equals("t1")){
            b.printName();
        }else{
            b.printValue();
        }
```

```
    }

}
```

1.print : printName , then wait for 5 seconds then print : printValue
2.print : printName then print : printValue
3.print : printName then wait for 5 minutes then print : printValue
4.Compilation succeed but Runtime Exception

Explanation :
B is the correct answer.

There is only one lock per object, if one thread has picked up the lock, no other thread can pick up the lock until the first thread releases the lock. In this case printName() is static , So lock is in class B not instance b, both method (one static and other no-static) can run simultaneously. A static synchronized method and a non static synchronized method will not block each other.

Question - 14

```
What is the output for the below code ?

public class B {
    public static synchronized void printName(){
        try{
               System.out.println("printName");
                Thread.sleep(5*1000);

            }catch(InterruptedException e){
            }
    }

    public static synchronized void printValue(){
               System.out.println("printValue");
    }
}



public class Test extends Thread{

    public static void main(String argv[]) throws Exception{
    Test t = new Test();
    Thread t1 = new Thread(t,"t1");
    Thread t2 = new Thread(t,"t2");
    t1.start();
    t2.start();
    }

    public void run(){
        if(Thread.currentThread().getName().equals("t1")){
            B.printName();
        }else{
            B.printValue();
        }
```

```
    }

}
```

1.print : printName , then wait for 5 seconds then print : printValue
2.print : printName then print : printValue
3.print : printName then wait for 5 minutes then print : printValue
4.Compilation succeed but Runtime Exception

Explanation :
A is the correct answer.

There is only one lock per object, if one thread has picked up the lock, no other thread can pick up the lock until the first thread releases the lock. In this case class B is locked when printName() method called because it is a static method. So, static printValue() method should wait.

Question - 15

```
What is the output for the below code ?

public class B {
    public synchronized void printName(){
        try{
                System.out.println("printName");
                 Thread.sleep(5*1000);

            }catch(InterruptedException e){
            }
    }

    public  void printValue(){
        synchronized(this) {
            System.out.println("printValue");
        }
    }
}



public class Test extends Thread{
    B b = new B();
    public static void main(String argv[]) throws Exception{
    Test t = new Test();
    Thread t1 = new Thread(t,"t1");
    Thread t2 = new Thread(t,"t2");
    t1.start();
    t2.start();
    }

    public void run(){
        if(Thread.currentThread().getName().equals("t1")){
            b.printName();
        }else{
```

```
            b.printValue();
        }
    }

}
```

1.print : printName , then wait for 5 seconds then print : printValue
2.print : printName then print : printValue
3.print : printName then wait for 5 minutes then print : printValue
4.Compilation succeed but Runtime Exception

Explanation :
A is the correct answer.

There is only one lock per object, if one thread has picked up the lock, no other thread can pick up the lock until the first thread releases the lock. public void printValue() { synchronized(this) { System.out.println("printValue"); } } is same in practical terms as public synchronized void printValue(){ System.out.println("printValue"); }

Question - 16

```
What is the output for the below code ?

class A extends Thread{
    int count = 0;
        public void run(){
            System.out.println("run");
            synchronized (this) {
                for(int i =0; i < 50 ; i++){
                    count = count + i;
                }
                notify();
        }
    }
}



public class Test{

    public static void main(String argv[]) {
        A a = new A();
        a.start();
        synchronized (a) {
            System.out.println("waiting");
            try{
                a.wait();
            }catch(InterruptedException e){

            }
            System.out.println(a.count);

        }
```

```
    }

}
```

1.waiting run 1225
2.waiting run 0
3.waiting run and count can be anything
4.Compilation fails

Explanation :
A is the correct answer.

a.wait(); put thread on wait until not get notifed. A thread gets on this waiting list by executing the wait() method of the target object. It doesn't execute any further instructions until the notify() method of the target object is called. A thread to call wait() or notify(), the thread has to be the owner of the lock for that object.

Question - 17

```
What is the output for the below code ?

class A extends Thread{
    int count = 0;
        public void run(){
            System.out.println("run");
            synchronized (this) {
                for(int i =0; i < 50 ; i++){
                    count = count + i;
                }
                notify();
        }
    }
}



public class Test{

    public static void main(String argv[]) {
        A a = new A();
        a.start();

            System.out.println("waiting");
            try{
                a.wait();
            }catch(InterruptedException e){

            }
            System.out.println(a.count);


    }

}
```

1.waiting run 1225
2.waiting run 0
3.waiting run and count can be anything
4.Compilation succeed but Runtime Exception

Explanation :
D is the correct answer.

wait() OR notify() OR notifyAll() must be called from within a synchronized context. A thread to call wait() or notify(), the thread has to be the owner of the lock for that object.

Question - 18

```
What is the output for the below code ?

class A extends Thread{
    int count = 0;
        public void run(){
            System.out.println("run");
            synchronized (this) {
                for(int i =0; i < 50 ; i++){
                    count = count + i;
                }
                notifyAll();
        }
    }
}



public class Test extends Thread{
    A a;
    public Test(A a){
        this.a = a;
    }

    public static void main(String argv[]) {
        A a = new A();
        Test t1 = new Test(a);
        Test t2 = new Test(a);
        t1.start();
        t2.start();
        a.start();
    }

    public void run(){
        synchronized (a) {
            System.out.println("waiting");
            try{
                a.wait();
            }catch(InterruptedException e){
```

```
            }
            System.out.println(a.count);

        }
        }

}
```

1.waiting waiting run 1225 1225
2.waiting waiting run 1225
3.waiting run 1225
4.Compilation succeed but Runtime Exception

Explanation :
A is the correct answer.

notifyAll() notify all the threads in waiting state. notifyAll() : Wakes up all threads that are waiting on this object's monitor.A thread waits on an object's monitor by calling one of the wait methods.

Question - 19

**An object that creates new threads on demand ?**

1.ThreadFactory
2.ThreadCreator
3.ThreadInitiator
4.None of the above

Explanation :
A is the correct answer.

An object that creates new threads on demand. Using thread factories removes hardwiring of calls to new Thread, enabling applications to use special thread subclasses, priorities, etc. The simplest implementation of this interface is just: class SimpleThreadFactory implements ThreadFactory { public Thread newThread(Runnable r) { return new Thread(r); } }

Question - 20

**Which of the below statement is correct ?**

1.Thread class implements Runnable interface
2.Thread class does not implements Runnable interface
3.Runnable interface extends Thread interface
4.None of the above are correct

Explanation :
A is the correct answer.

public class Thread extends Object implements Runnable

Question - 21

```
Thread class setPriority(int newPriority)  method allow _____________
MAX_PRIORITY ?
```

1.10
2.11
3.5
4.12

Explanation :
A is the correct answer.

Thread has MAX_PRIORITY = 10 and MIN_PRIORITY = 1 and priority should not be newPriority > MAX_PRIORITY || newPriority < MIN_PRIORITY

Question - 22

```
Thread class setPriority(int newPriority)  method allow _____________
MIN_PRIORITY ?
```

1.1
2.0
3.2
4.-1

Explanation :
A is the correct answer.

Thread has MAX_PRIORITY = 10 and MIN_PRIORITY = 1 setPriprity checks for if (newPriority > MAX_PRIORITY || newPriority < MIN_PRIORITY) { throw new IllegalArgumentException(); } where MAX_PRIORITY = 10 and MIN_PRIORITY=1

Question - 23

```
public static Map<Thread,StackTraceElement[]> getAllStackTraces()
method of Thread class _________________ ?
```

1.Returns a map of stack traces for all live threads

2.Returns a map of stack traces for all live and dead threads
3.Returns a map of stack traces for all live threads which are not related to current thread.
4.None of the above

Explanation :
A is the correct answer.

public static Map getAllStackTraces()Returns a map of stack traces for all live threads. The map keys are threads and each map value is an array of StackTraceElement that represents the stack dump of the corresponding Thread. The returned stack traces are in the format specified for the getStackTrace method

Question - 24

**Which of the below statement is true regarding Thread class ?**

1.getId() method of Thread class return thread ID is unique and remains unchanged during its lifetime
2.getId() method of Thread class return thread ID is unique and may change during its lifetime
3.getId() method of Thread class return thread ID of dead thread and it is not unique.
4.All of the above

Explanation :
A is the correct answer.

getId() method of Thread class return thread ID is unique and remains unchanged during its lifetime

Question - 25

**Which of the follwing is the signature of run() method in Runnable interface ?**

1.public String run();
2.public abstract void run();
3.public Object run();
4.None of the above

Explanation :
B is the correct answer.

public interface Runnable { public abstract void run(); }

Question - 26

**Which of the follwing is the signature of execute() method in Executor Interface ?**

1.void execute(Runnable command);
2.void execute(Thread command);
3.void execute(Object command);
4.None of the above

Explanation :
A is the correct answer.

public interface Executor { void execute(Runnable command); }

Question - 27

**java.util.concurrent package is used for ?**

1.Utility classes commonly useful in concurrent programming
2.Utility classes commonly useful in Applet programming
3.Utility classes commonly useful in JDBC programming
4.None of the above

Explanation :
A is the correct answer.

Utility classes commonly useful in concurrent programming

Question - 28

**Which statement is true about Lock Interface ?**

1.Lock implementations provide more extensive locking operations than can be obtained using synchronized methods and statements.
2.Lock implementations provide more extensive locking operations than cann't be obtained using synchronized methods and statements.
3.A lock is a tool for controlling access to a shared resource by multiple threads
4.1 and 3

Explanation :
D is the correct answer.

Lock implementations provide more extensive locking operations than can be obtained using synchronized methods and statements. A lock is a tool for controlling access to a shared resource by multiple threads

Question - 29

```
Is AtomicInteger threadsafe ?
```

1.yes it is threadsafe
2.not threadsafe
3.None of the above
4.None of the above

Explanation :
A is the correct answer.

An int value that may be updated atomically is AtomicInteger .

Question - 30

```
Is it true ?
java.util.concurrent.atomic package contains classes that support
lock-free thread-safe programming on single variables.
```

1.true
2.false
3.None of the above
4.None of the above

Explanation :
A is the correct answer.

java.util.concurrent.atomic package contains classes that support lock-free thread-safe programming on single variables.

Question - 31

```
Is all the methods of AtomicInteger is syncronized ?
```

1.Yes
2.No
3.Can't say
4.None of the above

Explanation :
B is the correct answer.

methods of AtomicInteger is not syncronized

Question - 32

**Can you pass a Thread object to Executor.execute?**

1.yes
2.No
3.Can't say
4.None of the above

Explanation :
A is the correct answer.

Thread implements the Runnable interface, so you can pass an instance of Thread to Executor.execute. However it doesn't make sense to use Thread objects this way. If the object is directly instantiated from Thread, its run method doesn't do anything. You can define a subclass of Thread with a useful run method ? but such a class would implement features that the executor would not use.

Question - 33

**In the bellow code**

```
public class SynchronizedCounterTest {
    private int c = 0;

    public synchronized void increment() {
        c++;
    }

    }
```

**Can two threads Simultaneously access the increment () method.**

1.No because method is synchronized
2.Yes
3.Can't say
4.None of the above

Explanation :
A is the correct answer.

only one thread at a time can access the synchronized method.

Question - 34

```
Thread.sleep(200);

where 200 is in ?
```

1.seconds
2.miliseconds
3.minutes
4.None of the above

Explanation :
B is the correct answer.

miliseconds

Question - 35

```
The __________ exception that sleep throws when another thread
interrupts the current thread while sleep is active ?
```

1.InterruptedException
2.IOException
3.SleepException
4.None of the above

Explanation :
A is the correct answer.

InterruptedException that sleep throws when another thread interrupts the current thread while sleep is active

Question - 36

```
What Thread.sleep() method do ?
```

1.causes the current thread to suspend execution for a specified period
2.causes the current thread dead.
3.causes the all threads dead.
4.None of the above

Explanation :
A is the correct answer.

Thread.sleep causes the current thread to suspend execution for a specified period. This is an efficient means of making processor time available to the other threads of an application or other applications that might be running on a computer system

Question - 37

**What Thread.interrupted() method do ?**

1.check for interrupted and interrupt status is cleared
2.check for interrupted only
3.There is no method like Thread.interrupted()
4.None of the above

Explanation :
A is the correct answer.

When a thread checks for an interrupt by invoking the static method Thread.interrupted, interrupt status is cleared

Question - 38

**What join() method do ?**

1.The join method allows one thread to wait for the completion of another
2.The join method allows one thread to terminate for the completion of another
3.causes the current thread dead.
4.None of the above

Explanation :
A is the correct answer.

The join method allows one thread to wait for the completion of another. If t is a Thread object whose thread is currently executing, t.join(); causes the current thread to pause execution until t's thread terminates. Overloads of join allow the programmer to specify a waiting period. However, as with sleep, join is dependent on the OS for timing, so you should not assume that join will wait exactly as long as you specify. Like sleep, join responds to an interrupt by exiting with an InterruptedException

Question - 39

**Will the bellow code compile ?**

```
public class HelloRunnable implements Runnable {

    public void run() {
        System.out.println("Hello from a thread!");
```

```
    }

    public static void main(String args[]) {
        (new Thread(new HelloRunnable())).start();
    }

}
```

1.Yes Compile
2.No Compile error
3.Can't say
4.None of the above

Explanation :
A is the correct answer.

The Runnable interface defines a single method, run, meant to contain the code executed in the thread

Question - 40

**What is Deadlock ?**

1.Deadlock describes a situation where two or more threads are blocked forever, waiting for each other
2.Deadlock never happend on Java
3.Deadlock never happend in single JVM
4.None of the above

Explanation :
A is the correct answer.

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other

Question - 41

**What is Starvation ?**

1.Starvation describes a situation where a thread is unable to gain regular access to shared resources and is unable to make progress
2.an object provides a synchronized method that often takes a long time to return. If one thread invokes this method frequently, other threads that also need frequent synchronized access to the same object will often be blocked.
3.Both are true
4.None of the above

Explanation :
C is the correct answer.

Starvation describes a situation where a thread is unable to gain regular access to shared resources and is unable to make progress. This happens when shared resources are made unavailable for long periods by "greedy" threads. For example, suppose an object provides a synchronized method that often takes a long time to return. If one thread invokes this method frequently, other threads that also need frequent synchronized access to the same object will often be blocked

Question - 42

```
Is this true ?
Thread class implements Runnable interface
```

1.true
2.false
3.Can't say
4.None of the above

Explanation :
A is the correct answer.

public class Threadextends Objectimplements Runnable

Question - 43

```
Which statement is correct about BlockingQueue ?

public interface BlockingQueue<E>extends Queue<E>
```

1.BlockingQueue implementations are thread-safe
2.BlockingQueue implementations are not thread-safe
3.BlockingQueue - No such interface defind in Java
4.None of the above

Explanation :
A is the correct answer.

BlockingQueue implementations are thread-safe

Question - 44

```
What is true about ConcurrentMap<K,V> ?
```

1.A Map providing additional atomic putIfAbsent, remove, and replace methods.
2.A Map can't be used for Thread programming.
3.It is not a Map
4.None of the above

Explanation :
A is the correct answer.

public interface ConcurrentMapextends MapA Map providing additional atomic putIfAbsent, remove, and replace methods.

Question - 45

```
Is V putIfAbsent(K key,V value) method of ConcurrentMap is like
if (!map.containsKey(key))
        return map.put(key, value);
   else
        return map.get(key);
except that the action is performed atomically ?
```

1.true
2.false
3.None of the above
4.None of the above

Explanation :
A is the correct answer.

V putIfAbsent(K key,V value) method of ConcurrentMap is like if (! map.containsKey(key)) return map.put(key, value); else return map.get(key); except that the action is performed atomically

Question - 46

```
Is AtomicInteger is threadsafe ?
```

1.true
2.false
3.None of the above
4.None of the above

Explanation :
A is the correct answer.

class SynchronizedCounter { private int c = 0; public synchronized void increment() { c++; } } and import java.util.concurrent.atomic.AtomicInteger; class AtomicCounter

{ private AtomicInteger c = new AtomicInteger(0); public void increment()
{ c.incrementAndGet(); } } is same

Question - 47

**What will happen when you attempt to compile and run the following code**

```
public class Test extends Thread{
    public static void main(String argv[]){
        Test  b = new Test();
        b.start();
    }
    public void run(){
        System.out.println("Running");
    }
}
```

1.Compilation and run but no output
2.Compilation and run with the output "Running"
3.Compile time error with complaint of no Thread target
4.Compile time error with complaint of no access to Thread package

Explanation :
B is the correct answer.

This is perfectly legitimate if useless sample of creating an instnace of a Thread and causing its run method to execute via a call to the start method. The Thread class is part of the core java.lang package and does not need any explicit import statement.

Question - 48

**What will happen when you attempt to compile and run the following code?**

```
public class B extends Thread{
public static void main(String argv[]){
B b = new B();
b.run();
}
public void start(){
for (int i = 0; i <10; i++){
System.out.println("Value of i = " + i);
}
}
}
```

1.A compile time error indicating that no run method is defined for the Thread class
2.A run time error indicating that no run method is defined for the Thread class
3.Clean compile and at run time the values 0 to 9 are printed out
4.Clean compile but no output at runtime

Explanation :
D is the correct answer.

This is a bit of a sneaky one as I have swapped around the names of the methods you need to define and call when running a thread. If the for loop were defined in a method called public void run() and the call in the main method had been to b.start() The list of values from 0 to 9 would have been output

Question - 49

```
What can cause a thread to stop executing?

1) The program exits via a call to System.exit(0);
2) Another thread is given a higher priority
3) A call to the thread's stop method.
4) A call to the halt method of the Thread class?
```

1.1 , 2 AND 3
2.1 , 2 AND 4
3.1 AND 4
4.None of the above

Explanation :
A is the correct answer.

can cause a thread to stop executing, not what will cause a thread to stop executing. Java threads are somewhat platform dependent and you should be carefull when making assumptions about Thread priorities.

Question - 50

```
Under what circumstances might you use the yield method of the Thread
class ?
```

1.To call from the currently running thread to allow another thread of the same or higher priority to run
2.To call on a waiting thread to allow it to run
3.To allow a thread of higher priority to run
4.To call from the currently running thread with a parameter designating which thread should be allowed to run

Explanation :
A is the correct answer.

Option 3 looks possible but there is no guarantee that the thread that grabs the cpu time will be of a higher priority. It will depend on the threading algorithm of the Java Virtual Machine and the underlying operating system

Question - 51

**Which of the following statements about threading are true**

**1) You can only obtain a mutually exclusive lock on methods in a class that extends Thread or implements runnable**

**2) You can obtain a mutually exclusive lock on any object**

**3) A thread can obtain a mutually exclusive lock on an object by calling a synchronized method on that object.**

**4) Thread scheduling algorithms are platform dependent**

1.2 , 3 and 4
2.1 and 2
3.1 , 3 and 4
4.None of the above

Explanation :
A is the correct answer.

statements 2, 4 and 4 are correct.

Question - 52

**Which of the following best describes the use of the synchronized keyword?**

1.Allows two process to run in paralell but to communicate with each other
2.Ensures only one thread at a time may access a method or object
3.Ensures that two or more processes will start and end at the same time
4.Ensures that two or more Threads will start and end at the same time

Explanation :
B is the correct answer.

Ensures only one thread at a time may access a method or object

Question - 53

**What is the output for the below code ?**

```
public class Test extends Thread{

    static String sName = "test";
    public static void main(String argv[]){
        Test t = new Test();
```

```
        t.printValue(sName);
        System.out.println(sName);

        }
        public void printValue(String sName){
                sName = sName + " new ";
                start();
        }
        public void run(){

        for(int i=0;i  < 3; i++){
                sName = sName + " " + i;

        }
        }

}
```

1.test
2.test new
3.test new test new
4.test new new new

Explanation :
A is the correct answer.

start() method does not call run() method because it is not thread call.

Question - 54

```
Which of the following are methods of the Thread class?

1) yield()
2) sleep(long msec)
3) go()
4) stop()
```

1.1 , 2 and 4
2.1 and 3
3.3 only
4.None of the above

Explanation :
A is the correct answer.

Check out the Java2 Docs for an explanation

Question - 55

```
"creating a new thread requires fewer resources than creating a new
process" Is this true ?
```

1.true
2.false
3.Can't say
4.None of the above

Explanation :
A is the correct answer.

creating a new thread requires fewer resources than creating a new process

Question - 56

**Which of the following not a method of the Thread class?**

1.yield()
2.sleep(long msec)
3.go()
4.stop()

Explanation :
C is the correct answer.

go() is not any method of Thread class

Question - 57

**What notifyAll() method do?**

1.Wakes up all threads that are waiting on this object's monitor
2.Wakes up one threads that are waiting on this object's monitor
3.Wakes up all threads that are not waiting on this object's monitor
4.None of the above

Explanation :
A is the correct answer.

notifyAll() : Wakes up all threads that are waiting on this object's monitor.A thread waits on an object's monitor by calling one of the wait methods.

Question - 58

**What wait(long timeout) method do ?**

1.Causes the current thread to wait until either another thread invokes the notify()

method or the notifyAll() method for this object, or a specified amount of time has elapsed.
2.No such method available
3.None of the above
4.None of the above

Explanation :
A is the correct answer.

public final void wait(long timeout) throws InterruptedException Causes the current thread to wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

Question - 59

Which of the following statements about this code are true?

```
class A  extends Thread{
    public void run(){
        for(int i =0; i < 2; i++){
            System.out.println(i);
        }
    }
}


public class Test{
    public static void main(String argv[]){
        Test t = new Test();
        t.check(new A(){});
    }
    public void check(A a){
        a.start();
        }

}
```

1.0 0
2.Compilation error, class A has no start method
3.0 1
4.Compilation succeed but runtime exception

Explanation :
C is the correct answer.

class A extends Thread means the anonymous instance that is passed to check() method has a start method which then calls the run method.

Question - 60

```
class A extends Thread{
    private String sname="";
    A(String s){
        sname = s;

    }
    public void run(){
        for(int i =0; i < 3 ; i++){
            try{
             sleep(5*1000);
            }catch(InterruptedException e){}

            yield();
            System.out.println(sname);
        }

    }
}

public class Test{
    public static void main(String argv[]){
    A a1 = new A("One");
    a1.run();
    A a2 = new A("Two");
    a2.run();

    }
}
```

1.Compile time error, class A does not import java.lang.Thread
2.One One One Two Two Two
3.One Two One Two One Two
4.Compilation succeed but no output at runtime

Explanation :
B is the correct answer.

If you call the run method directly it just acts as any other method and does not return to the calling code until it has finished executing.

Question - 61

```
Which of the below statement is true ?
```

1.methods of StringBuffer are synchronized
2.methods of StringBuilder are synchronized
3.methods of StringBuilder and StringBuffer are synchronized
4.None of the above

Explanation :
A is the correct answer.

methods of StringBuffer are synchronized.